

[平台服务](#)

模型迁移

 Copy page

从 OpenAI 模型快速迁移到智谱AI，享受便捷的搬家计划和兼容性支持

📘 本指南介绍了如何从 OpenAI 模型快速迁移到智谱AI，我们用实例展示这个过程，帮助您更好的完成迁移工作。

基础工具包

💡 OpenAI SDK 为我们提供了一个开箱即用的调用工具，对此，我们在后端兼容了 OpenAI 的所有 Endpoint，提供了便捷的迁移方式，仅需更换 `api_key` 与 `base_url`，就可以使用我们的模型。

切换API Endpoint

只需要简单修改两个参数即可完成迁移：

1 更换API Key

将 OpenAI 的 API Key 替换为智谱AI 的 API Key

2 更换Base URL

将 `base_url` 设置为智谱AI 的 API 地址

Python示例



```
>
from openai import OpenAI

client = OpenAI(
    api_key="your zhipuai api key",
    base_url="https://open.bigmodel.cn/api/paas/v4/"
)

response = client.chat.completions.create(
    model="glm-4",
    messages=[
        {"role": "system", "content": "你是一个聪明且富有创造力的小说作家"},
        {"role": "user", "content": "请你作为童话故事大王，写一篇短篇童话故事。"}
    ],
    top_p=0.7,
    temperature=0.9
)

print(response.choices[0].message)
```

使用智谱AI 官方 SDK

❗ 智谱AI 的部分功能需要您通过官方 SDK 进行调用，您可以通过 pypi 进行安装。

安装SDK

安装命令



```
pip install zhipuai
```

调用示例

```
from zhipuai import ZhipuAI

>
client = ZhipuAI(api_key="your-api-key") # 填写您自己的APIKey

response = client.chat.completions.create(
    model="glm-4", # 填写需要调用的模型名称
    messages=[
        {"role": "user", "content": "作为一名营销专家，请为智谱开放平台创作一个吸引人的slogan"},
        {"role": "assistant", "content": "当然，为了创作一个吸引人的slogan，请告诉我一些"},
        {"role": "user", "content": "智谱开放平台"},
        {"role": "assistant", "content": "智启未来，谱绘无限—智谱AI，让创新触手可及!"},
        {"role": "user", "content": "创建一个更精准、吸引人的slogan"}
    ],
)

print(response.choices[0].message)
```

开源框架支持

- ① 在智能体和相关任务框架的迁移方面，我们支持 LangChain 框架的快速迁移。我们提供了 langchain-zhipuai 的拓展工具，提供了对 BaseChatModel、Embeddings 的支持适配。

安装 LangChain 扩展

1 下载扩展包

下载地址: <https://github.com/MetaGLM/langchain-zhipuai/releases>

2 设置环境变量

使用前请设置环境变量 `ZHIPUAI_API_KEY`，值为智谱AI 的 API Key


```
from langchain_zhipuai.agents.zhipuai_all_tools.base import _get_assistants_tool
from langchain_zhipuai.chat_models import ChatZhipuAI
from langchain.agents import tool
from langchain.tools.shell import ShellTool
from pydantic.v1 import BaseModel, Extra, Field
from langchain import hub
from langchain_zhipuai.agents.all_tools_bind.base import create_zhipuai_tools_age
from langchain_zhipuai.agent_toolkits import BaseToolOutput
from langchain_zhipuai.agents.all_tools_agent import ZhipuAiAllToolsAgentExecutor

@tool
def shell(query: str = Field(description="The command to execute")):
    """Use Shell to execute system shell commands"""
    tool = ShellTool()
    return BaseToolOutput(tool.run(tool_input=query))

llm = ChatZhipuAI(api_key="") # 这里可以指定apikey

tools = [
    _get_assistants_tool(shell),
    {"type": "code_interpreter", "code_interpreter": {"sandbox": "none"}},
    {"type": "web_browser"},
    {"type": "drawing_tool"},
]

llm_with_all_tools = llm.bind(tools=tools)

prompt = hub.pull("zhipuai-all-tools-chat/zhipuai-all-tools-agent")
agent = create_zhipuai_tools_agent(
    prompt=prompt,
    llm_with_all_tools=llm_with_all_tools
)

agent_executor = ZhipuAiAllToolsAgentExecutor(
    agent=agent,
    tools=[shell],
    verbose=True,
    return_intermediate_steps=True,
```

```
)  
agent_executor.invoke({  
    "input": "你好",  
    "chat_history": [],  
})
```

兼容向量数据库

💡 我们提供了兼容 OpenAI 的 Embedding 调用方式，使用向量库相关的 Embeddings 时，您只需要替换 Embedding 的实例为 ZhipuAIEmbeddings。

向量化示例

向量化调用示例



```
from langchain_zhipuai.embeddings.base import ZhipuAIEmbeddings  
  
# Test zhipuai embeddings  
documents = ["foo bar"]  
embedding = ZhipuAIEmbeddings()  
output = embedding.embed_documents(documents)  
  
# len(output) == 1  
# len(output[0]) == 1024
```

迁移优势



无缝兼容

兼容 OpenAI API 接口，迁移成本低



性能优化

针对中文场景优化，提供更好的生成效果



成本优势 >

更具竞争力的定价策略，降低使用成本



本土化服务

本土化部署，提供更稳定的服务保障

< 内容安全

用户权益 >